

Meal Management System

A single-mess accounting system — meals, deposits, expenses, and member balances for one shared bachelor mess. Built as a portfolio project.

Stack: Django · Django REST Framework · django-rest-framework-simplejwt · PostgreSQL · React (frontend, separate app)

This README is the design of record for the Rev 3 rebuild. It supersedes the earlier Rev 1 (145-page spec) and Rev 2 (old repo) designs — see Design history at the bottom if you need the reasoning behind a decision.

Table of contents

1. Roles & permissions
2. Getting started
3. Core concepts
4. Data model
5. Application workflow
6. API reference
7. Build order
8. Design history

1. Roles & permissions

Two roles are stored in the database. A third is derived, never stored.

Role	Stored?	Granted by
Admin	Yes (<code>User.role = 'ADMIN'</code>)	Bootstrapped once via terminal command — see below
Member	Yes (<code>User.role = 'MEMBER'</code>)	Created by an Admin
Manager	No — derived	Holding an active <code>ManagerAssignment</code> on the currently open month

Permissions are capability-based, checked in code against a plain Python map — not a database table:

```
{}  
python
```

```

# apps/users/permissions.py
ROLE_CAPABILITIES = {
    "ADMIN": {
        "month.create", "month.open", "month.close",
        "manager.assign",
        "user.create", "user.view", "user.deactivate", "user.reset_password",
        "settings.manage", "adjustment.create", "audit.view",
    },
    "MEMBER": {
        "meal.submit", "meal.view_own",
        "deposit.submit", "deposit.view_own",
        "guest_meal.submit", "guest_meal.view_own",
        "report.view_own",
    },
}

# Added on top of base MEMBER capabilities for whoever is the active
# manager of the currently open month:
MANAGER_CAPABILITIES = {
    "meal.view_all",
    "deposit.approve",
    "guest_meal.approve",
    "expense.create", "expense.update", "expense.delete",
    "report.view_all",
}

```

`User.has_capability(code)` checks the role map, adds `MANAGER_CAPABILITIES` if the user is the active manager of the open month, and returns `True` unconditionally for `is_superuser`. Every view checks a capability code — never `request.user.role == "ADMIN"` directly.

Why Manager isn't a third stored role: it's scoped to one month at a time and can be reassigned mid-cycle. Storing it as a role creates the same contradiction this project hit in an earlier design pass — some code checking `role`, other code checking who's currently assigned — and it silently breaks the moment a manager handoff happens mid-month. Deriving it per-request from `ManagerAssignment` means there's exactly one source of truth.

2. Getting started



```
bash
git clone <repo-uri>
cd meal-management-system
python -m venv venv
venv\Scripts\activate    # Windows; source venv/bin/activate on macOS/Linux
pip install -r requirements.txt

# PostgreSQL required — partial unique indexes and check constraints
# used throughout this schema are silently skipped on SQLite.
python manage.py migrate

# The ONLY way an Admin account is ever created — no API endpoint for this:
python manage.py create_admin

python manage.py runserver
```

Everything past this point requires the Admin account to create Members and get the system running. There is no self-registration for anyone, by design (see Registration model below).

3. Core concepts

Meal unit

One lunch **or** one dinner for one person. A day with both selected is **two** units, not one.

Countable meal units



countable meal units = member meal units + approved guest meal units

This is the denominator for the meal rate. Member meals carry no approval status — only guest meals do — so "approved" only ever modifies "guest meal units."

Month lifecycle — three states, not two



PLANNED —open→ OPEN —close→ CLOSED

State	Meals / guest meals	Deposits / expenses / adjustments
-------	---------------------	-----------------------------------

PLANNED	Accepted	Rejected
---------	----------	----------

OPEN	Accepted	Accepted
------	----------	----------

CLOSED	Read-only	Read-only
--------	-----------	-----------

Only one month may be **OPEN** at a time (database-enforced). Any number may be **PLANNED**.

Why three states: members submit meals for *tomorrow*. If a month can only be **OPEN** or **CLOSED**, then on the last evening of the month, tomorrow's date belongs to a month that doesn't exist yet — and it can't be created **OPEN**, because the current

month still is. Every month's day one would collect zero meals. Creating next month as `PLANNED` ahead of time fixes this without breaking the single-open-month rule, which only ever governed money.

Balance chain

Balances carry forward across months. Sign convention: **positive = member owes the mess**, negative = advance.

```
┌
|
| closing_balance = opening_balance + total_cost + adjustment_total
|               - approved_deposit_total
|
| opening_balance(month N) = closing_balance(month N-1)
|               (zero if no prior month, or a new joiner)
```

Rate calculation and rounding

```
┌
|
| rate      = ROUND(total_expense / countable_meal_units, 4)
| member_cost = ROUND(meal_units × rate, 2)      [half-up, per member]
| residual   = total_expense - Σ member_cost
```

The residual (a few poysha from rounding) is posted as an `Adjustment` to the member with the highest meal-unit count that month (ties broken by lowest member id) — so total member costs reconcile to total expenses **exactly**, and the adjustment is visible in the ledger rather than silently absorbed.

Corrections are adjustments, never edits

An approved deposit, an approved guest meal, or a rate-rounding residual is never fixed by editing the original record. It's corrected by posting an `Adjustment` (`DEBIT` or `CREDIT`) with a mandatory reason. This keeps every approved record exactly as it was approved, with the correction visible as its own ledger entry.

Similarly, a meal submitted wrong after the cutoff isn't edited in place — it's corrected via an `Adjustment` against the member's balance.

Nothing is hard-deleted

Users are deactivated, never deleted. Expenses are soft-deleted (`is_deleted` / `deleted_at` / `deleted_by` / `delete_reason`) — the default queryset excludes them, so every total stays correct automatically. `AuditLog` is append-only; there is no update or delete endpoint for it anywhere.

Registration model

- **Admin** — bootstrapped once via `python manage.py create_admin`. No API endpoint creates an Admin, ever.
- **Member** — created only by an Admin, via `POST /api/users/`. The system generates a temporary password server-side, returns it once in the response, and forces a password change on first login (`must_change_password`). The Admin relays the password to the new member out of band (message, in person).
- **Manager** — never registered. Any existing Member becomes Manager by being assigned via `PUT /api/months/{id}/manager/`.

No self-registration exists anywhere in this system. It's a closed accounting tool for one specific mess, not a public product — every member is someone the Admin already knows and is vouching for.

4. Data model

10 entities.

Entity	Purpose
User	Account + stored role (ADMIN/MEMBER)
Month	Accounting period, 3-state lifecycle
ManagerAssignment	Who is Manager, for which month, when
MonthMember	Roster while open; frozen per-member result once closed
Meal	Daily lunch/dinner selection per member
GuestMeal	Guest meal requests with approval workflow
Deposit	Member payments with approval workflow
Expense	Mess purchases, soft-deletable
Adjustment	Manual debit/credit corrections
SystemSetting	Singleton config (deadline, timezone, currency)
AuditLog	Append-only action history

<details> <summary>Full field reference (click to expand)</summary>

User id, username (unique, immutable), email (unique), password, role (ADMIN/MEMBER, default MEMBER), phone, is_active (default True), must_change_password (default True), created_at

Month id, name (unique), start_date, end_date, status (PLANNED/OPEN/CLOSED, default PLANNED), final_meal_rate (decimal 12,4, null), final_total_expense (decimal 12,2, null), rounding_residual (decimal 12,2, null), closed_at, closed_by, created_by, created_at — partial unique index: one status='OPEN' at a time. Check constraint: start_date < end_date.

ManagerAssignment id, month (FK), user (FK), assigned_by (FK), assigned_at, unassigned_at (null) — partial unique index: at most one row per month where unassigned_at IS NULL.

MonthMember id, month (FK), member (FK), unique(month, member), joined_on, left_on (null), opening_balance (decimal 12,2, default 0), lunch_count, dinner_count, meal_units, guest_lunch_count, guest_dinner_count, guest_meal_units, meal_rate_applied (decimal 12,4, null), meal_cost, guest_meal_cost, adjustment_total, total_cost, approved_deposit_total, closing_balance (decimal 12,2, null), finalized_at (null)

Meal id, member (FK), month (FK, server-resolved from meal_date), meal_date, lunch (bool, default False), dinner (bool, default False), created_at, updated_at — unique together: (member, meal_date).

GuestMeal id, member (FK, host), month (FK), meal_date, lunch_quantity, dinner_quantity (at least one > 0), status (PENDING/APPROVED/REJECTED, default PENDING), rejection_reason, submitted_at, processed_at (null), processed_by (null) — partial unique index: (member, meal_date) unique excluding REJECTED rows.

Deposit id, member (FK), month (FK), amount (decimal 12,2, > 0), payment_method (CASH/BKASH/NAGAD/BANK), transaction_reference (required if not CASH), payment_date, status (PENDING/APPROVED/REJECTED, default PENDING), rejection_reason, submitted_at, processed_at (null), processed_by (null) — only against an OPEN month.

Expense id, month (FK), created_by (FK), category (Grocery/Gas/Electricity/Water/Internet/Cleaning/Maintenance/Others), amount (decimal 12,2, > 0), expense_date, description, is_deleted (default False), deleted_at, deleted_by, delete_reason — soft delete only; default queryset excludes is_deleted=True.

Adjustment id, month (FK), member (FK), direction (DEBIT/CREDIT), amount (decimal 12,2, > 0), reason (required), source_entity_type, source_entity_id, created_by (FK, Admin only)

SystemSetting (singleton, pk=1) mess_name, meal_submission_deadline (time), timezone (default Asia/Dhaka), currency (default BDT)

AuditLog id, actor (FK, null for system), action, entity_type, entity_id, description, created_at

</details>

5. Application workflow

First-time setup



1. Admin runs `create\_admin` from the terminal
2. Admin logs in → POST /api/auth/login/
3. Admin creates each Member → POST /api/users/ (returns a one-time temp password)
4. Admin hands each temp password to its member out of band
5. Each Member logs in and is forced to change their password
6. Admin creates the first Month → POST /api/months/ (starts PLANNED)
7. Admin opens it → POST /api/months/{id}/open/ (roster snapshot, balances carried forward)
8. Admin assigns a Manager for the month → PUT /api/months/{id}/manager/

Monthly cycle, once running



Members → submit daily meals (POST /api/meals/) before the cutoff
→ submit deposits (POST /api/deposits/) as they pay
→ request guest meals (POST /api/guest-meals/) as needed

Manager → approves/rejects deposits and guest meals
→ adds/edits/soft-deletes expenses
→ monitors meal counts and running totals via the dashboard

Admin (a few days before month end) → creates next month as PLANNED, so tomorrow's meal submissions have somewhere to land

Admin, at month end → closes the month → POST /api/months/{id}/close/
→ validates: no PENDING deposits/guest meals, countable units > 0, expenses > 0, a Manager was assigned
→ computes final rate, member costs, allocates the rounding residual, freezes every MonthMember row
→ promotes the next PLANNED month to OPEN

Admin, if a mistake is found post-close → reopens → POST /api/months/{id}/reopen/, fixes the underlying data, closes again

One member's month, end to end



Day 1-29 Member checks lunch/dinner daily before the deadline.
Member deposits money in installments as convenient — each
one sits PENDING until the Manager approves it.
Member optionally requests a guest meal for a visiting friend.

Month end Admin closes the month. The member's MonthMember row freezes:

$\text{meal_units} \times \text{final_rate} = \text{meal_cost}$
 $\text{guest_meal_units} \times \text{final_rate} = \text{guest_meal_cost}$
+ any Adjustments this month
– approved deposits this month
+ opening_balance (last month's leftover)
= this month's closing_balance

Next month opening_balance = that closing_balance. The cycle repeats.

6. API reference

Resource-oriented — one route per resource, visibility enforced by queryset filtering against the caller's capabilities, not by role-prefixed URLs.



/api/auth/ login · refresh · logout · me · password/change
/api/users/ list · create · detail · deactivate · reactivate · reset-password
/api/months/ list · create · detail · open · close
/{id}/members/ roster (open) or frozen result (closed)
/{id}/manager/ assign manager
/api/meals/ list (own, or all if manager+) · create (upsert)
/api/guest-meals/ list · create · approve · reject
/api/deposits/ list · create · approve · reject
/api/expenses/ list · create · update · delete (soft)
/api/adjustments/ list · create (Admin only)
/api/settings/ retrieve · update
/api/audit-logs/ list (Admin only)
/api/dashboard/ capability-shaped composite view
/api/reports/ monthly · members · balances

Endpoint detail

Method & path	Capability	Notes
POST /auth/login/	public	Returns <code>access</code> , <code>refresh</code> , <code>user</code> (with <code>must_change_password</code>)
POST /auth/refresh/	public	Rotates refresh token, blacklists the old one
POST /auth/logout/	authenticated	Blacklists the presented refresh token
GET /auth/me/	authenticated	Profile + full current capability list
POST /auth/password/change/	authenticated	Clears <code>must_change_password</code>
GET /users/	<code>user.view</code>	Filters: <code>role</code> , <code>is_active</code> , <code>search</code>

Method & path	Capability	Notes
POST /users/	user.create	Returns a one-time <code>temporary_password</code> field
POST /users/{id}/deactivate/	user.deactivate	Blocked if the user is the active manager of the open month
POST /users/{id}/reactivate/	user.deactivate	—
POST /users/{id}/reset-password/	user.reset_password	New temp password, re-arms <code>must_change_password</code>
POST /months/	month.create	Created as <code>PLANNED</code>
POST /months/{id}/open/	month.open	Writes roster, carries balances forward
POST /months/{id}/close/	month.close	Transactional, row-locked, computes finals
GET /months/{id}/members/	month.view	Roster or frozen result
PUT /months/{id}/manager/	manager.assign	Closes prior assignment, opens new one
GET /meals/	meal.view_own / meal.view_all	Scope follows capability
POST /meals/	meal.submit	Upsert on <code>(member, meal_date)</code> ; blocked past deadline
GET /guest-meals/	guest_meal.view_own / guest_meal.approve	—
POST /guest-meals/	guest_meal.submit	Resubmittable after rejection

Method & path	Capability	Notes
POST /guest-meals/{id}/approve/	guest_meal.approve	From PENDING only
POST /guest-meals/{id}/reject/	guest_meal.approve	Reason required
POST /deposits/	deposit.submit	Only against OPEN month
POST /deposits/{id}/approve/	deposit.approve	From PENDING only
POST /deposits/{id}/reject/	deposit.approve	Reason required
POST /expenses/	expense.create	Only against OPEN month
DELETE /expenses/{id}/	expense.delete	Soft delete, reason required
POST /adjustments/	adjustment.create	Admin only, open month only
GET /dashboard/	authenticated	Payload shaped by capabilities
GET /reports/balances/	report.view_all / report.view own	Indicates ESTIMATED vs FINAL
GET /audit-logs/	audit.view	Admin only, read-only

Error convention: every error response includes a machine-readable `code` field (e.g. `deadline_passed`, `month_closed`, `already_approved`, `password_change_required`) alongside the message, so the frontend can branch on `code` rather than matching on prose.

7. Build order

1. Models + migrations for all 10 entities (PostgreSQL required — partial indexes, check constraints)
2. `ROLE_CAPABILITIES` / `MANAGER_CAPABILITIES` map + `HasCapability` permission class
3. Auth: login, refresh, logout, me, password change, `must_change_password` gate
4. `create_admin` management command
5. Month lifecycle: create → open (roster + carry-forward) → close (calculation + residual allocation) — **write the reconciliation test here**
6. User management endpoints (Admin creates/deactivates/resets Members)
7. Meals + guest meals (deadline enforcement, resubmit-after-rejection)
8. Deposits + expenses + adjustments + approval flows
9. Dashboard + reports + audit logging
10. React client

The reconciliation test is the one to get right before anything else builds on top of it: close a month with an odd number of members, a rate that doesn't divide evenly, one member who left mid-month, and one who joined late. Assert that the sum of every member's `total_cost` plus the rounding adjustment equals `total_expense` exactly, and that each member's `closing_balance` becomes the correct `opening_balance` in the next month.

Design history

This is Rev 3 of the design. Earlier revisions are kept for context, not as current instructions:

- **Rev 1** — a 145-page spec document covering the full original requirements analysis, UML diagrams, and business rules. Used a single daily decimal `meal_count` field instead of separate lunch/dinner booleans, and a plain `role` field with no Manager scoping.
- **Rev 2** — resolved 20 architectural defects found in Rev 1 (month-boundary deadlock, missing per-member close snapshot, no balance carry-forward, hard deletes, undefined rounding, and more). Introduced the 3-state month, the `MonthMember` snapshot

pattern, and Adjustment-based corrections — all of which carry forward into Rev 3. Built out in a separate repo with 12 entities, including a `Notification` entity and database-seeded `Role`/`Permission` tables, both simplified away in Rev 3.

- **Rev 3** (this document) — a leaner rebuild in a new repo, keeping every correctness fix from Rev 2 that prevents a real bug, while dropping Notifications and DB-seeded permissions in favor of a plain code-level capability map. 10 entities instead of 12.